INVENTORY MANAGEMENT SYSTEM

Complete Solution Design & Build Documentation

A centralized, real-time, multi-warehouse stock operations platform

RECEIPTS	DELIVERY	TRANSFER	ADJUST
Incoming Stock	Outgoing Stock	Internal Moves	Stock Corrections

01 · Problem Analysis

1.1 The Core Problem

Businesses today manage inventory using fragmented, manual methods — physical registers, Excel sheets, WhatsApp messages, and disconnected tools. This creates a chain of critical failures that silently drain revenue, time, and customer trust.

Pain Point	Real-World Impact
No single source of truth	5 people, 5 different stock numbers. No one knows the real count.
Zero audit trail	When stock goes missing, there's no way to trace who moved it, when, or where.
No low-stock alerts	Products run out mid-production — causing emergency orders and missed deadlines.
Manual data entry errors	Typos and mis-keyed quantities silently corrupt stock levels over time.
No multi-location tracking	Stock in Warehouse A is invisible to staff in Warehouse B. Transfers are verbal.
Reactive, never proactive	Problems discovered after they've caused delays, losses, or customer complaints.

1.2 Who Suffers & How

Inventory Managers	Warehouse Staff
• No real-time visibility into stock	• Waste time hunting for items
• Reports take hours to compile manually	• No mobile tools for counting/scanning
• Cannot enforce approval workflows	• Cannot confirm completed transfers
• Cannot identify receipt/delivery bottlenecks	• Physical counts take entire days

1.3 The Business Cost of No IMS

- Lost revenue from stockouts and missed delivery deadlines
- Excess capital locked in overstock due to poor visibility
- 2-3 days of labour per quarter for manual counting operations
- Compliance risk from inability to produce stock audit trails
- Customer churn when orders cannot be fulfilled on time

02 · Requirements Specification

2.1 Functional Requirements

Authentication & Access Control

- Email + password sign up and login with JWT tokens
- OTP-based password reset (6-digit code delivered via email)
- Role-based access: Inventory Manager vs Warehouse Staff
- Profile management: name, avatar, role, warehouse assignment

Dashboard & KPIs

- 6 KPI cards: Total Products, Low Stock, Out of Stock, Pending Receipts, Pending Deliveries, Scheduled Transfers
- Real-time alert banner for low-stock items with one-click navigation
- Filterable operation feed by: type, status, warehouse, date range, category
- Export dashboard snapshot to PDF

Product Management

- Create/edit/archive products with: Name, SKU, Category, Unit of Measure, Min stock level, Reorder qty
- Stock availability per location in real-time
- Hierarchical category management
- Reordering rules: min qty, max qty, reorder point, preferred supplier
- SKU search with smart autocomplete and barcode/QR generation

Receipts — Incoming Stock (+stock)

- Create with: reference number, supplier, scheduled date, receiving warehouse
- Status lifecycle: Draft → Confirmed → Ready → Done / Backorder
- Partial receipt support: receive partial, auto-create backorder for remainder
- Validate → stock_quants auto-increase + immutable ledger entry created

Delivery Orders — Outgoing Stock (-stock)

- Pick-Pack-Ship 3-step workflow with individual confirmations
- Stock availability check before confirmation — warns if insufficient
- Status: Draft → Waiting → Ready → Done / Cancelled / Backorder
- Validate → stock decreases + reservation released + ledger entry created

Internal Transfers — Location to Location

- Move stock between any two locations: warehouse, rack, zone, bin
- Atomic transaction: source decreases and destination increases simultaneously
- Total org stock unchanged — only location updated in stock_quants

Stock Adjustments — Fixing Mismatches

- Enter physically counted quantity → system calculates delta automatically
- Reason codes: Damaged, Theft, Expiry, Miscounting, Other
- Manager approval required before adjustment is applied
- Full audit entry created on every approved adjustment

Stock Ledger & Move History

- Every stock movement creates an immutable append-only ledger entry
- Fields: Timestamp, User, Operation type, Product, Qty moved, Source, Destination, Doc reference
- Full product trace: select any SKU → see complete movement history
- Export to Excel and PDF with applied filters

2.2 Non-Functional Requirements

Category	Requirement	Target
Performance	Dashboard load time	< 2 seconds on standard connection
Performance	Stock update after validation	Real-time propagation < 500ms
Scalability	Products per organisation	Up to 10,000 SKUs without degradation
Scalability	Concurrent users	50+ simultaneous warehouse staff
Security	Authentication method	JWT + bcrypt hashing, HTTPS only
Security	Data isolation	Multi-tenant by organisation (row-level)
Usability	Mobile responsiveness	Full functionality on tablet and mobile
Usability	Staff onboarding time	Productive within 30 minutes
Reliability	Monthly uptime target	99.5% SLA

03 · Solution Design

3.1 System Architecture

The IMS follows a clean 3-tier architecture: a React SPA frontend, a Node.js REST API backend, and a PostgreSQL database. Real-time updates are delivered via Socket.io. Redis caches dashboard aggregations for sub-second response times.

Frontend Layer	Backend Layer	Data Layer
React 18 + TypeScript	Node.js + Express + TypeScript	PostgreSQL 15
Tailwind CSS	Prisma ORM	Redis (dashboard caching)
React Query (data sync)	JWT + bcrypt auth	S3-compatible file storage
Recharts (charts)	Socket.io (real-time push)	Automated daily backups
React Hook Form + Zod	Zod request validation	Row-level security

3.2 Database Schema

Table	Key Fields	Purpose
users	id, email, role, org_id	Authentication & role management
organisations	id, name, settings	Multi-tenant data isolation
warehouses	id, name, address, org_id	Physical warehouse locations
locations	id, name, warehouse_id, type	Racks, zones, bins within warehouse
products	id, name, sku, category_id, uom, min_qty	Master product catalog
categories	id, name, parent_id	Hierarchical product categories
stock_quants	product_id, location_id, qty, reserved_qty	Current stock per product per location
receipts	id, supplier, status, warehouse_id, date	Incoming goods documents
delivery_orders	id, customer, status, warehouse_id, date	Outgoing goods documents
stock_moves	id, product_id, src_loc, dest_loc, qty	Individual product lines on operations
stock_ledger	id, move_id, user_id, qty, balance, timestamp	Immutable audit log of all movements
adjustments	id, product_id, reason, delta, approved_by	Physical count corrections

reorder_rules	product_id, min_qty, max_qty, reorder_point	Automated reorder triggers
alerts	id, type, product_id, seen, created_at	Low stock and system notifications

3.3 Core Inventory Flow

Every stock movement follows one universal pattern — a Stock Move record is created, confirmed, validated, and an immutable ledger entry is appended. This guarantees a perfect audit trail for every transaction in the system.

1	2	3	4	5
Create Doc	**Add Lines**	**Confirm**	**Validate**	**Ledger**
Draft created	Products + qty	Reserve stock	Apply movement	Immutable log

3.4 Operations Detail

Receipt — Incoming Goods (+stock)

Trigger: Goods arrive from a vendor or supplier

- Create receipt with supplier name, scheduled date, receiving warehouse
- Add product lines: product SKU, expected quantity, unit of measure
- Confirm: receipt moves to Ready — all product lines locked
- Staff inputs actual received quantities (may be less than expected)
- Validate: stock_quants.qty increases, immutable ledger entry created
- If partial: backorder auto-created for remaining undelivered quantity

Delivery Order — Outgoing Goods (-stock)

Trigger: Customer order needs to be picked, packed, and shipped

- Create delivery order with customer, source warehouse, scheduled date
- System checks availability: warns if any product lacks sufficient stock
- Confirm: stock reserved (reserved_qty increases) to prevent double allocation
- Pick: warehouse staff gets guided picking list sorted by warehouse location
- Pack: items packed and verified against the delivery order quantities
- Validate: stock_quants.qty decreases, reservation released, ledger entry created

Internal Transfer — Location to Location

Trigger: Stock must move between two locations within the organisation

- Create transfer: source location, destination location, product lines
- System checks source location has sufficient stock before confirmation
- Validate: atomic DB transaction — source qty decreases, dest qty increases
- Total organisation stock unchanged — only location distribution updated
- Ledger records both -qty at source AND +qty at destination as separate entries

Stock Adjustment — Fixing Discrepancies

Trigger: Physical count reveals a mismatch from system records

- Select product and specific location to count
- Enter physically counted quantity
- System calculates: delta = counted_qty minus current_recorded_qty
- Staff selects reason code: Damaged / Theft / Expiry / Miscounting / Other
- Manager reviews justification and approves or rejects the adjustment
- On approval: stock_quants updated, immutable ledger entry created with reason

04 · UI / UX Design

4.1 Navigation Structure

Section	Sub-Pages	Who Can Access
Dashboard	KPIs, operation feed, alert bar	Manager + Staff
Products	List, Create/Edit, Categories, Reorder Rules, Stock by location	Manager edit / Staff view
Receipts	List, Create, Detail, Validate, Backorders	Manager + Staff
Deliveries	List, Create, Pick list, Pack, Validate	Manager + Staff
Transfers	List, Create, Validate	Manager + Staff
Adjustments	List, Create, Approve (Manager only)	Manager approve / Staff create
Move History	Full ledger with filters and export	Manager only
Settings	Warehouses, Locations, Users, Alert config	Manager only
Profile	Personal info, password, preferences	All users

4.2 Dashboard Layout

6 KPI Cards (responsive grid, real-time via Socket.io)

- Total Products in Stock — count of active SKUs with qty > 0
- Low Stock Items — products below their configured minimum threshold
- Out of Stock Items — products with total qty = 0 across all locations
- Pending Receipts — receipts in Ready or Waiting state
- Pending Deliveries — delivery orders in Ready or Waiting state
- Internal Transfers Scheduled — transfers created but not yet validated

Dynamic Filter Bar (sits above the operation feed)

- Document type tabs: All / Receipts / Deliveries / Transfers / Adjustments
- Status filter: Draft / Waiting / Ready / Done / Cancelled / Backorder
- Warehouse filter: multi-select dropdown listing all configured warehouses
- Date range: date-from and date-to picker
- Category filter: product category dropdown
- Global search: full-text search by SKU, product name, or reference number

4.3 Winning UX Differentiators

These features separate this IMS from every other submission in the competition:

- Smart Stock Reservation: Confirming a delivery immediately reserves that stock so it cannot be double-allocated to another concurrent order.
- Guided Picking List: Deliveries generate a picking list sorted by physical warehouse location (zone, rack, shelf), minimising staff walking distance.
- Barcode/QR Integration: Every product gets a printable QR code. Staff scan with phone camera to instantly pull up the product and create stock movements.
- Batch Validation: Select multiple pending operations and validate them all at once — critical for busy receiving docks processing multiple deliveries simultaneously.
- Reorder Alert with One-Click Draft: Low-stock alerts include a 'Create Receipt' button that opens the new receipt form pre-filled with the flagged product.
- Visual Stock Sparkline: Each product page shows a 30-day stock-level chart from the ledger, revealing consumption patterns and anomalies at a glance.
- Keyboard-First: Global search (Ctrl+K), keyboard navigation, and shortcut keys for common operations make power users extremely efficient.
- Export Everywhere: Every list view exports to CSV and PDF in one click, including the full stock ledger, receipts list, and stock valuation summary.

05 · Competitive Advantage

5.1 Feature Matrix vs Alternatives

Feature	Excel/Manual	Basic App	This IMS
Real-time stock levels	No	Sometimes	Yes — instant push
Complete audit trail	No	Partial	Yes — every movement
Multi-warehouse support	Manual only	Limited	Yes — unlimited
Smart low-stock alerts	None	Basic email	Real-time + action button
Barcode / QR scanning	No	No	Yes — camera scan
Guided pick-pack-ship	No	No	Yes — step-by-step
Stock reservation system	No	No	Yes — auto on confirm
Partial receipts + backorders	No	No	Yes — fully automatic
Role-based access control	No	Basic	Manager vs Staff roles
One-click export (CSV + PDF)	Manual	No	Yes — every list view
Real-time dashboard updates	No	No	Yes — Socket.io push
Mobile responsive UI	No	Partial	Yes — fully responsive

06 · Master Build Prompt

Use this prompt to generate the complete IMS application code:

Give this entire prompt to Claude, Cursor, or any AI coding assistant. It contains every module, endpoint, page, and design requirement needed to build a production-ready IMS.

```
BUILD PROMPT: INVENTORY MANAGEMENT SYSTEM (IMS)
================================================

Build a production-ready, full-stack Inventory Management System.

TECH STACK:
  Frontend : React 18 + TypeScript + Vite + Tailwind CSS + React Query
             + React Hook Form + Zod + Recharts + React Router v6
  Backend  : Node.js + Express + TypeScript + Prisma ORM + JWT + bcrypt
             + Socket.io + Zod validation
  Database : PostgreSQL 15 (via Prisma)
  Cache    : Redis (dashboard aggregations, 30s TTL)
  Storage  : S3-compatible (product images, document attachments)

BACKEND MODULES & API ENDPOINTS:

1. AUTH
   POST /auth/register     { name, email, password, role: MANAGER|STAFF }
   POST /auth/login        { email, password } -> { accessToken, refreshToken }
   POST /auth/refresh      { refreshToken } -> new accessToken
   POST /auth/forgot-password  { email } -> sends 6-digit OTP
   POST /auth/reset-password   { email, otp, newPassword }
   GET  /auth/me           -> current user profile

2. PRODUCTS
   GET    /products              List with filters: category, status, search
   POST   /products              Create product
   GET    /products/:id          Product detail
   PUT    /products/:id          Update product
   DELETE /products/:id          Archive (soft delete)
   GET    /products/:id/stock    Stock qty per location
   GET    /products/:id/history  Stock ledger for this product (last 90 days)
   CRUD   /categories            Category management
   CRUD   /reorder-rules         Reorder rule management

3. WAREHOUSES & LOCATIONS
   CRUD /warehouses             Warehouse management
   CRUD /locations              Location management (linked to warehouse)

4. RECEIPTS (Incoming Stock)
   GET  /receipts               List with filters
   POST /receipts               Create receipt { supplier, warehouse_id, lines[] }
   GET  /receipts/:id           Receipt detail with lines
   PUT  /receipts/:id           Update draft receipt
   POST /receipts/:id/confirm   Status: DRAFT -> CONFIRMED
   POST /receipts/:id/validate  Status: READY -> DONE
```

```
                                Increases stock_quants, creates ledger entries
                                Creates backorder if partial (done_qty < demand_qty)

5. DELIVERY ORDERS (Outgoing Stock)
   GET  /deliveries             List with filters
   POST /deliveries             Create delivery { customer, warehouse_id, lines[] }
   GET  /deliveries/:id         Delivery detail + pick list
   PUT  /deliveries/:id         Update draft delivery
   POST /deliveries/:id/confirm Status: DRAFT -> WAITING -> READY
                                Check availability; reserve stock (reserved_qty++)
   POST /deliveries/:id/validate Status: READY -> DONE
                                Decrease qty, release reservation, create ledger entries
   POST /deliveries/:id/cancel  Status -> CANCELLED; release all reservations

6. INTERNAL TRANSFERS
   GET  /transfers              List with filters
   POST /transfers              Create { src_location_id, dest_location_id, lines[] }
   GET  /transfers/:id          Transfer detail
   POST /transfers/:id/confirm  DRAFT -> READY
   POST /transfers/:id/validate READY -> DONE
                                Atomic: src.qty--, dest.qty++, create 2 ledger entries

7. ADJUSTMENTS
   GET  /adjustments            List (managers see all, staff see own)
   POST /adjustments            Create { product_id, location_id, counted_qty, reason }
   GET  /adjustments/:id        Adjustment detail
   POST /adjustments/:id/approve  PENDING -> APPROVED; update stock, create ledger
   POST /adjustments/:id/reject   PENDING -> REJECTED

8. STOCK LEDGER
   GET /stock-ledger            All entries, filters: product_id, location_id,
                                operation_type, date_from, date_to, user_id
                                Paginated, sortable, exportable

9. DASHBOARD
   GET /dashboard               Returns:
     { totalProducts, lowStockCount, outOfStockCount, pendingReceipts,
       pendingDeliveries, scheduledTransfers, recentOperations[20],
       alerts[], lowStockProducts[] }
   Use Redis cache, 30s TTL. Bust cache after any stock_quants change.

10. ALERTS
    After every stock_quants update, check min_stock_level.
    Create alert if qty < min_stock_level AND no existing unseen alert.
    Push via Socket.io to all MANAGER users in same organisation.
    GET  /alerts       My alerts
    PUT  /alerts/:id/seen  Mark as seen

11. EXPORTS
    GET /receipts/export?format=csv|pdf
    GET /deliveries/export?format=csv|pdf
    GET /stock-ledger/export?format=csv|pdf
    GET /products/export?format=csv|pdf  (includes current stock per location)
    Use pdfkit for PDF, csv-writer for CSV.

DATABASE: Prisma Schema Models
```

```
  User, Organisation, Warehouse, Location, Product, Category,
  StockQuant (qty + reserved_qty), Receipt, ReceiptLine,
  DeliveryOrder, DeliveryLine, Transfer, TransferLine,
  Adjustment, StockMove, StockLedger, ReorderRule, Alert

MIDDLEWARE:
  - authMiddleware: verify JWT, attach req.user
  - roleCheck(MANAGER): reject STAFF on protected routes
  - validateBody(schema): Zod validation on all POST/PUT
  - rateLimiter: 100 req/min per IP
  - errorHandler: consistent error response format

FRONTEND PAGES (React Router v6):
  /auth/login                  Login form
  /auth/register               Registration form
  /auth/forgot-password        OTP request + reset flow
  /dashboard                   KPI cards + alert bar + operation feed
  /products                    Product list (search, filter, sort, export)
  /products/new                Create product form
  /products/:id                Product detail: info + stock map + sparkline + history
  /receipts                    Receipts list with status filters
  /receipts/new                Create receipt wizard
  /receipts/:id                Receipt detail + validate button
  /deliveries                  Deliveries list
  /deliveries/new              Create delivery form
  /deliveries/:id              Pick list view + pack confirmation + validate
  /deliveries/:id/pick         Guided picking checklist sorted by location
  /transfers                   Transfers list
  /transfers/new               Create transfer form
  /transfers/:id               Transfer detail + validate
  /adjustments                 Adjustments list
  /adjustments/new             Create adjustment form
  /adjustments/:id             Detail + manager approve/reject
  /history                     Stock ledger: full filters + export
  /settings/warehouses         Warehouse CRUD
  /settings/locations          Location CRUD per warehouse
  /settings/users              User management (MANAGER only)
  /profile                     User profile + password change

DESIGN SYSTEM:
  Primary color: Teal #0D9488
  Dark text: #111827 | Body text: #374151 | Muted: #6B7280
  Status badge colors:
    DRAFT=gray | WAITING=amber | READY=blue | DONE=green
    CANCELLED=red | BACKORDER=orange
  Layout: Left sidebar (240px) with icon + label nav items
  Mobile: Bottom tab bar replaces sidebar on screens < 768px
  Every list: search bar + filter dropdowns + sortable columns
              + pagination (50 per page) + export button
  Toast notifications for all create/update/validate actions
  Confirm dialog before every destructive or irreversible action

SPECIAL FEATURES (implement all of these):

A. STOCK RESERVATION
   stock_quants has: qty (total) and reserved_qty (held for deliveries)
```

```
   Available to pick = qty - reserved_qty
   Show both values on product detail and delivery lines

B. GUIDED PICK LIST
   GET /deliveries/:id/pick-list returns lines sorted by:
   warehouse -> zone -> rack -> shelf
   Frontend renders as a checklist with checkbox per line
   Staff tick items as they pick them (local state, not saved)

C. PRODUCT SPARKLINE
   On product detail page, show a Recharts LineChart
   Data: daily stock balance from stock_ledger for last 30 days
   Show horizontal dashed line at min_stock_level
   Color: green when above min, red when below

D. REAL-TIME DASHBOARD
   Socket.io: server emits 'dashboard:update' after every stock change
   Client updates KPI cards without page refresh
   Server emits 'alert:new' when low stock detected
   Client shows toast + updates alert bell count

E. GLOBAL SEARCH (Ctrl+K / Cmd+K)
   Command palette overlay
   Search products by name or SKU
   Search receipts/deliveries by reference number
   Navigate directly to result on selection

F. REORDER ALERT ACTION
   Low stock alert cards on dashboard have:
   [View Product] button -> navigate to product detail
   [Create Receipt] button -> navigate to /receipts/new
   with product pre-selected in line items

G. STATUS MACHINE ENFORCEMENT
   All status transitions validated on backend
   Invalid transitions return 400 with clear error message
   Frontend disables action buttons for invalid states

FILE STRUCTURE:
  /client
    /src/components/{ui,layout,forms,tables}
    /src/pages/{auth,dashboard,products,receipts,deliveries,
                transfers,adjustments,history,settings,profile}
    /src/hooks      (useAuth, useProducts, useReceipts, useSocket...)
    /src/lib        (api.ts, queryClient.ts, utils.ts)
    /src/types      (all TypeScript interfaces)
    /src/store      (auth store with Zustand)
  /server
    /src/routes     (one file per module)
    /src/controllers
    /src/middleware
    /src/services   (stockService, alertService, exportService)
    /prisma         (schema.prisma + seed.ts + migrations)
  docker-compose.yml   (PostgreSQL 15 + Redis 7)
  README.md            (setup, env vars, seed instructions)
```

```
SEED DATA (prisma/seed.ts):
  Organisation: 'Acme Manufacturing Ltd'
  Warehouses: Main Store, Production Floor, Finished Goods Store
  Locations: Main Store has zones A1-A5, B1-B5. Production has P1-P10.
  Categories: Raw Materials, Packaging, Spare Parts, Electronics, Consumables
  Products: 50 realistic products with SKUs (RM-001 to RM-020, etc.)
  Receipts: 10 completed receipts from the last 30 days
  Deliveries: 8 completed + 2 pending (one in WAITING, one in READY)
  Transfers: 3 completed + 2 pending
  Adjustments: 3 approved adjustments with different reason codes
  Users:
    admin@acme.com   password: demo1234  role: MANAGER
    staff@acme.com   password: demo1234  role: STAFF

Start by generating:
  1. prisma/schema.prisma (all models with relations)
  2. server/src/middleware/ (auth, role, validation, error)
  3. All backend routes and controllers
  4. React app scaffolding (router, layout, auth context)
  5. All frontend pages and components
  6. docker-compose.yml and README.md

Build this as a real production application. No shortcuts.
```

This document covers the complete solution for the IMS problem statement. Every requirement addressed. Every flow documented. Build prompt ready to execute.